Constructing Cybersecurity Knowledge Graphs from Unstructured Intelligence

Andrian Danar Perdana, Muhammad Argya Vityasy, Muhammad Bagas Prayoga, Rayhan Firdaus Ardian, and Sultan Devino Suyudi

No Institute Given

Abstract. Summarize the full CSKG project in 150–250 words. Highlight (i) the five unstructured source types, (ii) ontology or schema integration, (iii) pipeline technology choices (ML/LLM/RML), (iv) knowledge graph statistics, and (v) representative use-cases.

Keywords: Cybersecurity knowledge graph · Unstructured data · Semantic web · Ontology alignment · Threat intelligence

1 Introduction

Cybersecurity Knowledge Graphs (CSKGs) play a crucial role in supporting threat analysis, incident investigation, and data-driven decision-making in modern security operations. By representing relationships among key cybersecurity entities—such as malware, threat actors, vulnerabilities, attack techniques, and infrastructures—CSKGs enable integrated reasoning across diverse sources of threat intelligence. However, existing CSKG construction efforts predominantly rely on structured data sources, including standardized repositories such as CVE, CAPEC, and ATT&CK. While these sources are well-organized and machine-readable, they capture only a portion of the rapidly evolving cybersecurity landscape.

In practice, most threat intelligence emerges first in the form of unstructured Cyber Threat Intelligence (CTI), such as incident reports, security blogposts, technical analyses, advisories, and threat-hunting articles. These unstructured artifacts contain rich contextual information—details about attacker behavior, evolving tactics, newly observed indicators, and deep narrative insights that are often missing from structured datasets. However, their unstructured nature makes automated extraction and integration into CSKGs significantly challenging. The variability of writing styles, domain-specific terminology, and implicit relationships within free text creates a substantial gap between the available intelligence in the field and what current CSKG systems are capable of capturing.

This project aims to address this gap by focusing on the construction of a CSKG from unstructured CTI sources. Leveraging techniques from Machine Learning, Large Language Models (LLMs), and semantic mapping frameworks, the project seeks to extract cybersecurity entities and relations from textual data and align them with existing cybersecurity ontologies. The primary contributions of this work are as follows:

1. Identification and curation of multiple types of unstructured CTI sources relevant to cyber threat analysis.
2. Development of an automated pipeline for entity extraction, normalization, and CSKG construction.
3. Ontology-based semantic mapping to ensure consistency and interoperability with existing CSKGs.
4. Evaluation of the constructed graph, including statistical analysis and identification of missing or erroneous elements.
5. Demonstration of real-world use cases, such as threat hunting, indicator enrichment, and exploration of attack patterns.

By transforming unstructured CTI into a coherent and interconnected knowledge graph, this project aims to produce a more comprehensive and adaptive representation of cybersecurity knowledge—one that reflects current threat dynamics and supports more effective analytical and operational workflows.

2 Related Work

2.1 Knowledge Graph Construction in Cybersecurity

Several efforts have focused on constructing Cybersecurity Knowledge Graphs (CSKGs) to integrate and structure diverse threat intelligence sources. One of the most established initiatives is the SEPSES CSKG, which unifies multiple structured cybersecurity datasets—including CVE, CAPEC, CWE, ATT&CK, and NVD—into a consolidated semantic graph [3]. The SEPSES framework demonstrates the value of applying ETL workflows and semantic integration for supporting tasks such as vulnerability assessment, attack pattern exploration, and cross-dataset reasoning. However, its reliance on structured and semi-structured data limits its ability to capture the rich and dynamic information contained within unstructured Cyber Threat Intelligence (CTI) reports.

Beyond SEPSES, several ontologies contribute foundational modeling constructs for cybersecurity. The Unified Cybersecurity Ontology (UCO) provides a harmonized and interoperable vocabulary that bridges heterogeneous cybersecurity schemas and supports the representation of events, indicators of compromise (IoCs), threat actors, and system behaviors [1]. Meanwhile, the MALOnt ontology focuses specifically on malware behavior and system interactions, offering a structured framework for modeling how malicious software propagates and affects digital infrastructures [5]. These ontologies, while comprehensive, assume standardized and well-formatted inputs, and therefore do not directly address the extraction or integration of knowledge from unstructured text.

More recent work extends CSKG construction into specialized domains. The ICS-SEC Knowledge Graph applies semantic integration techniques to the industrial control systems (ICS) security domain, combining ICS-specific ontologies, structured IoC feeds, and mappings to standards such as STIX [4]. Although ICS-SEC expands the applicability of CSKGs to operational technology (OT) environments, it—like SEPSES—remains reliant on structured intelligence sources

and does not include automated extraction from narrative CTI artifacts such as analyst blogposts or incident reports.

Collectively, these initiatives highlight significant progress in structured-data CSKG development while revealing a persistent gap: the lack of mechanisms for incorporating knowledge from unstructured CTI sources. This limitation motivates the need for automated extraction and ontology-driven integration techniques to enrich CSKGs with information derived from natural-language threat intelligence.

2.2 Unstructured Threat Intelligence Extraction

A substantial portion of actionable cyber threat intelligence is expressed in unstructured textual formats, including incident reports, security advisories, malware analyses, and technical blogposts. Extracting structured knowledge from these free-text sources has therefore become an important research direction within cybersecurity natural language processing (NLP) and information extraction (IE).

Early approaches primarily relied on rule-based extraction, using handcrafted patterns or regular expressions to identify basic indicators of compromise (IoCs) such as IP addresses, URLs, file hashes, and registry keys. While effective for simple artifacts, such methods lack generalizability and are unable to capture the deeper behavioral and relational information present in narrative CTI.

Later studies introduced traditional machine learning models, particularly named entity recognition (NER) models trained on limited cybersecurity corpora, to extract domain-specific entities such as malware names, CVE identifiers, threat actors, and attack techniques. Although these approaches improved robustness, they remained constrained by the scarcity of annotated CTI datasets and the high linguistic variability of cybersecurity text.

Recent advancements leverage deep neural networks and Large Language Models (LLMs) to extract both entities and relations from unstructured CTI. These approaches enable identification of richer semantic concepts such as attacker behavior, exploitation chains, and tactics, techniques, and procedures (TTPs). For example, ontology-guided extraction frameworks such as OntoLogX employ LLMs and semantic alignment mechanisms to transform cybersecurity log data and free-text narratives into structured knowledge graph triples [2]. Some pipelines further incorporate mapping to standardized cybersecurity vocabularies such as STIX to normalize extracted intelligence.

Despite these developments, end-to-end pipelines that incorporate unstructured CTI extraction into full CSKG construction remain rare. Challenges persist due to inconsistent terminology, implicit relationships, narrative reporting structures, and limited labeled data. As a result, most existing CSKGs remain grounded in structured datasets. Addressing these limitations, this project aims to develop a pipeline capable of extracting, normalizing, and semantically integrating unstructured CTI into a CSKG that complements existing structured resources such as SEPSES and ICS-SEC.

3 Proposed Approach

The conceptual framework implements an automated, event-driven pipeline to construct a **Cybersecurity Knowledge Graph (CSKG)** from diverse unstructured data sources. This method addresses the challenge of synthesizing actionable threat intelligence by employing a **Large Language Model (LLM)** for efficient Information Extraction (IE) and mapping the results to a formal, extensible cybersecurity ontology. The resulting graph is persisted in a **Virtuoso SPARQL endpoint** for complex, semantic querying.

3.1 Unstructured Source Landscape

Sources were strategically selected to ensure comprehensive coverage, integrating **real-time indicators** with **in-depth technical analysis**. This variety minimizes the blind spots inherent in relying on singular data streams.

Table 1. Selected Unstructured Data Sources and Justification

Category	Justification
Security News Blogs (RSS)	Provides high velocity, continuous updates on emerging threats and vulnerabilities.
Vendor Attack Reports	Offers validated, in-depth technical analysis (often in semi-structured PDF/blog format) of adversarial campaigns.
CVE Descriptions (NVD)	Supplies authoritative, standardized textual details crucial for vulnerability mapping.
Threat Intel Tweets	Captures real-time, often leading-edge, short-form indicators from researcher communities.
Malware Analysis Reports	Delivers crucial technical breakdowns of malicious software behavior and internal structure.

3.2 Ontology Strategy

A **hybrid ontology** model is utilized to ensure semantic rigor while allowing for domain-specific extension.

- **Reused Ontology: STIX 2.1** is adopted as the primary semantic model, providing standardized classes (e.g., `STIX:ThreatActor`, `STIX:Malware`) and relationship types.
- **Reused Modules:** Explicit **cross-KG linking** targets the **SEPSES CVE Knowledge Graph**. Recognized CVE identifiers are mapped directly to their established SEPSES URIs.
- **Custom Namespace:** The namespace `http://group2.org/cskg/` (prefix `cskg`) is used for the named graph persistence and for custom entities lacking a direct STIX equivalent.

– **Alignment:** A critical `RELATIONSHIP_MAP` bridges LLM output (plain-English verbs) and formal, constrained **STIX relationship properties** (`STIX:uses`), ensuring ontological consistency.

3.3 End-to-End Pipeline

The architecture is a microservice-based, event-driven pipeline orchestrated via Docker Compose and **Redis** messaging queues.

Table 2. CSKG Pipeline Stages and Key Technologies

Stage	Component	Key Technology/Function
Collection	`producer` (`scraper.py`)	Scrapes and normalizes input data (RSS feeds).
Preprocessing	`producer` / **Redis**	Filters duplicates using a `seen_urls` set.
Information Extraction (IE)	`extractor`	**LLM Prompting** (Google **Gemini** via **LangChain**).
Triple Generation	`graph_builder`	**rdflib** transforms JSON to RDF; maps to STIX/custom ontology.
Entity Alignment	`graph_builder` (Logic)	Maps recognized CVEs to external **SEPSES URIs**.
Validation/Storage	**Virtuoso**	Persistence via SPARQL `INSERT DATA` queries.

4 Implementation

The pipeline was realized as an event-driven, microservice architecture, orchestrated via `docker-compose.yml`. The entire system is modular, using a **Redis** instance as the central message bus for queuing articles and extractions.

4.1 Data Acquisition and Normalization

Data acquisition is handled by the `producer` component (`pipeline/scraper.py`), implemented in Python.

– **Crawlers/APIs:** The primary acquisition method involves scraping **RSS feeds** from high-volume security news providers (e.g., TheHackerNews, BleepingComputer). This establishes a continuous, low-latency source ingestion.
– **Cleansing Steps:** A crucial normalization step involves **duplicate checking**. The `producer` utilizes a Redis `set` (`seen_urls`) to store hashes of processed URLs, preventing redundant ingestion and processing of previously seen content.
– **Language Detection:** Implicitly handled by focusing initial scraping on known English-language threat intelligence blogs; no explicit language model is run.

4.2 Information Extraction Components

Extraction is performed by the `extractor` worker, which listens to the Redis queue for new articles.

- **Extraction Tooling:** The core of the IE component is a **LangChain** pipeline leveraging a **Google Gemini LLM**.
- **Template Prompts:** The LLM is supplied with structured prompts designed to guide output toward specific cyber entities (`ThreatActor`, `Malware`, `Vulnerability`) and their directed relationships (e.g., `uses`, `targets`).
- **RDF Generation and Relation Extraction: Relation Extraction (RE)** and **Named Entity Recognition (NER)** are performed simultaneously by the LLM, which outputs a single JSON object structured according to **Pydantic schemas**.
- **Confidence Handling:** Explicit numerical confidence scores are **not** generated. Instead, reliability is managed through the use of strict **Pydantic output parsers**, which reject malformed or hallucinated outputs that do not conform to the predefined entity/relationship schema.

4.3 Knowledge Graph Construction

The `graph_builder` worker handles the semantic transformation and persistence of the structured extractions.

- **RDF Generation:** RDF triples are generated using a custom Python script (`pipeline/build_kg.py`) utilizing the `rdflib` library. This logic explicitly transforms the Pydantic-validated JSON into triples, rather than relying on RML mappings.
- **Namespace Conventions:** The primary namespace for classes and predicates is the industry-standard **STIX 2.1**. A custom namespace, `http://group2.org/cskg/`, is employed for the specific named graph and any entities lacking direct STIX classification.
- **SEPSES Entity Linking:** A key feature is the entity alignment logic within `build_kg.py` that detects CVE identifiers and maps them to pre-defined **SEPSES CVE URIs**, facilitating external KG linkage.
- **SPARQL Endpoint Deployment:** The persistent store is an **OpenLink Virtuoso** container. Triples are committed live using **SPARQL** `INSERT DATA` queries, ensuring a continuously updating knowledge graph.

5 Use-Case Applications

The constructed CSKG provides a semantic layer over unstructured threat intelligence, enabling complex, graph-based queries that accelerate defensive operations. We present three real-world application scenarios.

5.1 Use-Case 1: Attack Surface Monitoring

The CSKG accelerates proactive monitoring by tracing newly disclosed adversarial tactics (`STIX:AttackPattern` or `STIX:Malware`) back to vulnerable assets within an organization's inventory.

- **Process:** An analyst queries the KG to find all `STIX:AttackPattern` nodes that are related to a newly observed technique (e.g., using `stix:uses` or `stix:exploits`) a specific `STIX:Vulnerability`.
- **Value:** By linking the vulnerability node (which is often linked externally to the **SEPSES CVE KG**) to internal asset management data, the organization can immediately identify and prioritize assets directly exposed to the latest tactics.

5.2 Use-Case 2: Incident Response Triage

During incident triage, the CSKG significantly reduces the Mean Time To Contain (MTTC) by providing instantaneous context to discovered indicators.

- **Process:** A security appliance logs a suspicious artifact (e.g., an IP address, file hash), which is mapped to a `STIX:Indicator` node in the graph. The analyst queries outward from this node.
- **Acceleration:** The KG instantly resolves the Indicator to its associated `STIX:Malware`, the responsible `STIX:ThreatActor`, and the primary `STIX:Report` (source article). This single query provides the full contextual profile needed to inform immediate containment and eradication strategies (e.g., IoC blocking, behavior hunting).

5.3 Use-Case 3: Vulnerability Intelligence Linking

The graph structure facilitates automated linking of disparate intelligence elements related to a single zero-day or vulnerability bulletin.

- **Process:** When a `STIX:Vulnerability` (identified by its CVE ID and linked to SEPSES) is registered, the KG connects it to:
  1. Threat actors/malware observed to actively exploit it (`stix:exploits`).
  2. Indicators associated with that exploitation (`stix:indicator`).
  3. Mitigation steps mentioned in the source `STIX:Report` or related documents.
- **Integration:** This connectivity allows vulnerability management systems to ingest a cohesive profile linking risk, exploitation evidence, and appropriate defensive actions from a single semantic source.

6 Evaluation and Discussion

Report statistics, quality checks, and lessons learned.

6.1 Knowledge Graph Summary Statistics

Include Table 3 covering node/edge counts, class/property coverage, alignment ratios, etc.

Table 3. Example placeholders for CSKG statistics. Replace with actual values.

Metric	Value
Number of triples	TBD
Unique entities	TBD
Source documents processed	TBD
Links to SEPSES CSKG	TBD
Average extraction confidence	TBD

6.2 Error Analysis and Missing Knowledge

Discuss pipeline failure modes, ontology gaps, ambiguous entities, or coverage issues.

6.3 Discussion

Interpret the evaluation, trade-offs between automation and curation, and future improvements.

7 Conclusion

This work successfully designed and implemented an automated pipeline for constructing a **Cybersecurity Knowledge Graph (CSKG)** from unstructured threat intelligence sources.

7.1 Achievements

The primary achievement is the fully operational, event-driven data pipeline, integrating multiple microservices (scraper, extractor, builder) using Redis as a message bus. This system successfully:

- Established continuous ingestion of unstructured text from dynamic sources (e.g., RSS feeds).
- Utilized a **Google Gemini LLM** via `LangChain` to perform high-fidelity entity and relationship extraction.
- Persisted the extracted semantic data in a **Virtuoso SPARQL endpoint**, conforming to the RDF data model.
- Demonstrated semantic querying capability via a dedicated `FastAPI` interface.

7.2 Contributions

The core contributions advance automated CSKG construction by:

- Proposing a **hybrid ontology approach**, mapping LLM output directly to the industry-standard **STIX 2.1** while preserving flexibility via a custom namespace.
- Implementing a critical **cross-KG linking layer** that automatically integrates discovered CVEs with the external **SEPSES CVE Knowledge Graph**, significantly enriching the threat context.
- Validating a methodology for generating structured triples from noisy, heterogeneous cybersecurity text with high scalability potential.

7.3 Future Work

Future efforts will focus on enhancing the depth and breadth of the CSKG:

- **Wider Data Coverage:** Expanding source collection to include PDF reports, social media scraping, and deep web forums.
- **Entity Resolution:** Implementing a robust entity disambiguation layer to merge nodes (e.g., `APT29` and `APT 29`) and improve graph coherence.
- **Multilingual Support:** Extending the LLM extraction prompts and processing to natively support non-English threat reports.
- **Temporal Analysis:** Incorporating time series data and temporal reasoning into the graph model to track threat evolution.

Appendix (Optional)

Use appendices for supplementary figures, expanded ontology diagrams, detailed prompts, or SPARQL query listings.

Acknowledgments. Acknowledge collaborators, funding, or datasets.

Disclosure of Interests. State competing interests, if any, or explicitly note the absence of conflicts.

References

1. Unified cybersecurity ontology (uco). `https://unifiedcyberontology.org/ontology/`, accessed: 2025-11-18
2. Cotti, L., Drago, I., Rula, A., Bianchini, D., Cerutti, F.: Ontologx: Ontology-guided knowledge graph extraction from cybersecurity logs with large language models. arXiv preprint **arXiv:2510.01409** (2025)
3. Kiesling, E., Ekelhart, A., Kurniawan, K., Ekaputra, F.J.: The sepses knowledge graph: An integrated resource for cybersecurity. In: The Semantic Web – ISWC 2019 (Lecture Notes in Computer Science). pp. 198–214. LNCS, Springer (2019). https://doi.org/10.1007/978-3-030-30796-7_13

4. Kurniawan, K., Kiesling, E., Winkler, D., Ekelhart, A.: The ICS-SEC knowledge graph: An integrated cybersecurity resource for industrial control systems. In: Proceedings of ISWC 2024. pp. 153–170 (2024)
5. Rastogi, N., Dutta, S., Zaki, M.J., Gittens, A., Aggarwal, C.: Malont: An ontology for malware threat intelligence. arXiv preprint **arXiv:2006.11446** (2020)